

Deep Learning

①

Training Deep Neural Networks.

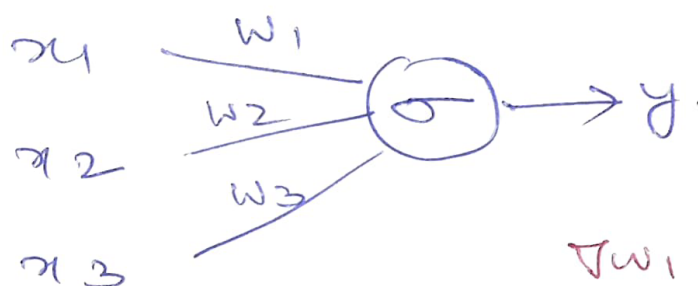


$$W = W - \lambda \nabla W$$

$$\nabla W = \frac{\partial J(W)}{\partial W}$$

or $\nabla W = W - \lambda [a_k - d_k]$

Wider network.



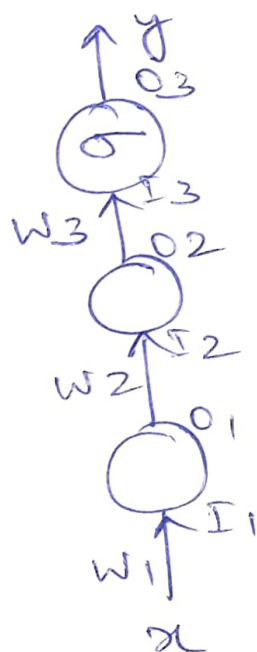
$$W_1 = W_1 - \lambda \nabla W_1$$

$$W_2 = W_2 - \lambda \nabla W_2$$

$$W_3 = W_3 - \lambda \nabla W_3$$

$$\nabla W_1 = W_1 - \lambda [a_k - d_k]$$

Deeper network.



In Backpropagation,

$$\frac{\partial J(W_1)}{\partial W_1} = \frac{\partial J(W)}{\partial o_3} \frac{\partial o_3}{\partial I_3}$$

$$\frac{\partial I_3}{\partial o_2} \frac{\partial o_2}{\partial I_2} \frac{\partial I_2}{\partial o_1}$$

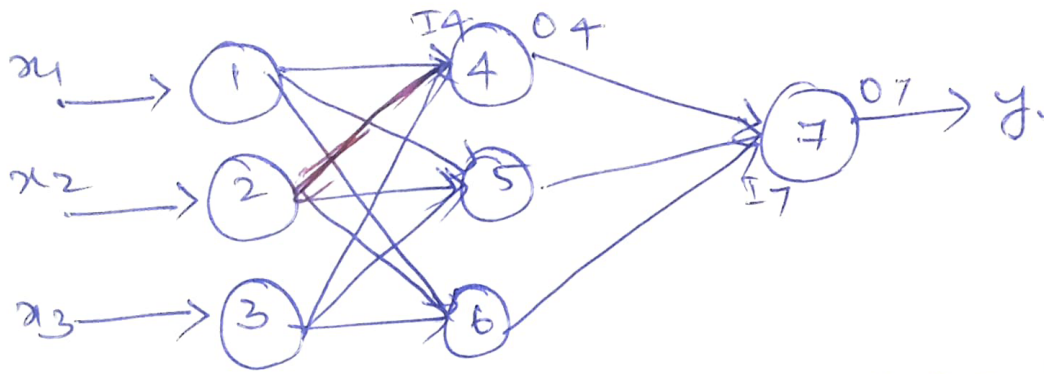
$$\frac{\partial o_1}{\partial W_1}$$

In general,

$$\nabla W = \frac{\partial J(W)}{\partial (y)} * \dots * I_{i-1}$$

What happens if we have a N/w which is deep and wide?

(2)



$$\nabla W_{24} = \frac{\partial J_W}{\partial W_{24}} = \frac{\partial J_W}{\partial O_7} \frac{\partial O_7}{\partial I_7} \frac{\partial I_7}{\partial O_4} \frac{\partial O_4}{\partial I_4} \frac{\partial I_4}{\partial W_{24}}$$

In General — Backpropagation.

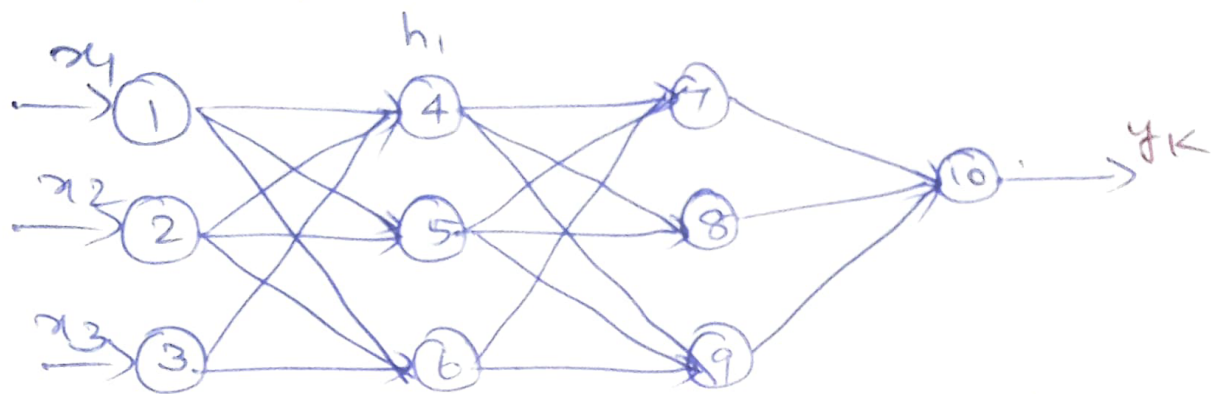
↳ Rumelhart et. al in 1986.

→ The term Backpropagation is not very successful in case of deep and wider networks.

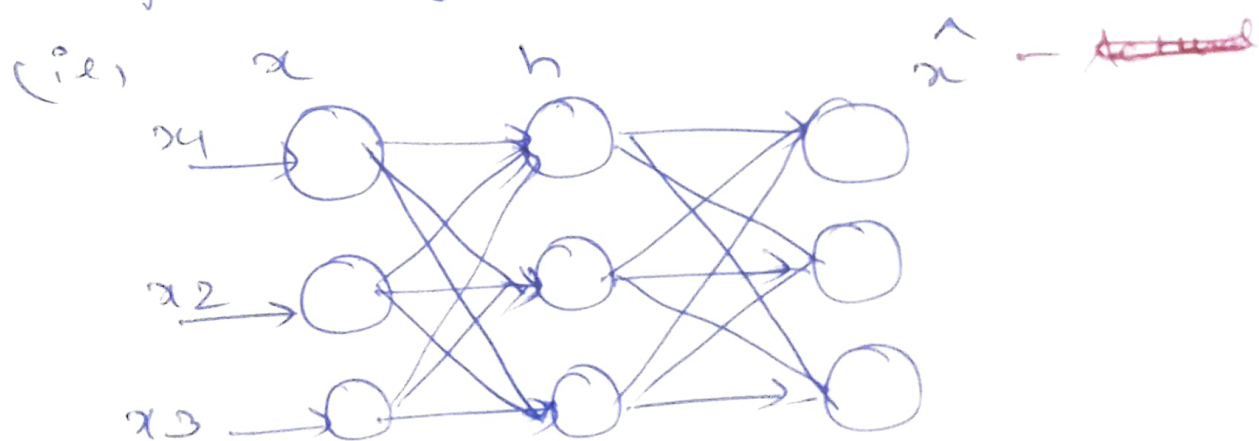
→ Even after no of epochs, the training didn't converge.

Unsupervised Training (3)

Hinton and Salakhutdinov et al in 2006 proposed an Unsupervised training.



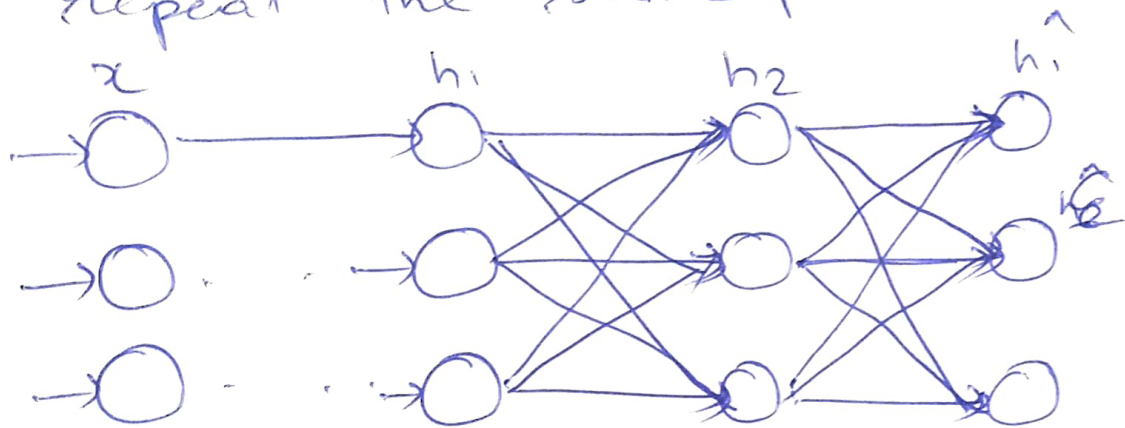
- Let us focus on first two layers of the network (x and h)
- train the weights between these two layers using an unsupervised objective.



$$\min \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

- Refers to unsupervised objective as it doesn't involve (y) and only uses the input data (x).

- (4)
- end of this step, the weights in layer 1 are trained such that h_1 captures an abstract representation of input x .
 - Fix the weights in layer 1 and repeat the same process with layer 2.



$$\min_m \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (h_{1,ij} - \hat{h}_{ij})^2$$

Finally, all the layers with layers are pre-training, the whole network will be trained using the task specific objective.

Thus

$$\min_m \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Why does the unsupervised pre-training works better?

→ Is it because of better optimization?

→ Is it because of better regularization?

Is it because of better optimization? ⁽⁵⁾

* No, the unsupervised - layer wise training can be used in case of small capacity networks.

Is it because of better Regularization?

→ Regularization Constrains the weights to certain regions of parameter space.

→ L1 regularization - Constrains most weights to be zero.

→ L2 regularization - prevents most weights from taking large values.

* Indeed, pre-training Constrains the weights to be in only certain regions of the parameter space.

* Specifically, it Constrains the weights to be in regions where the characteristics of the data are captured well.

$$\text{Minimize } \sum_{i=1}^m \sum_{j=1}^n (x_{ij} - y_{ij})^2 \quad (6)$$

Unsupervised objective :-

$$= \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (x_{ij} - \hat{x}_{ij})^2$$

whereas, supervised objective.

$$= \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

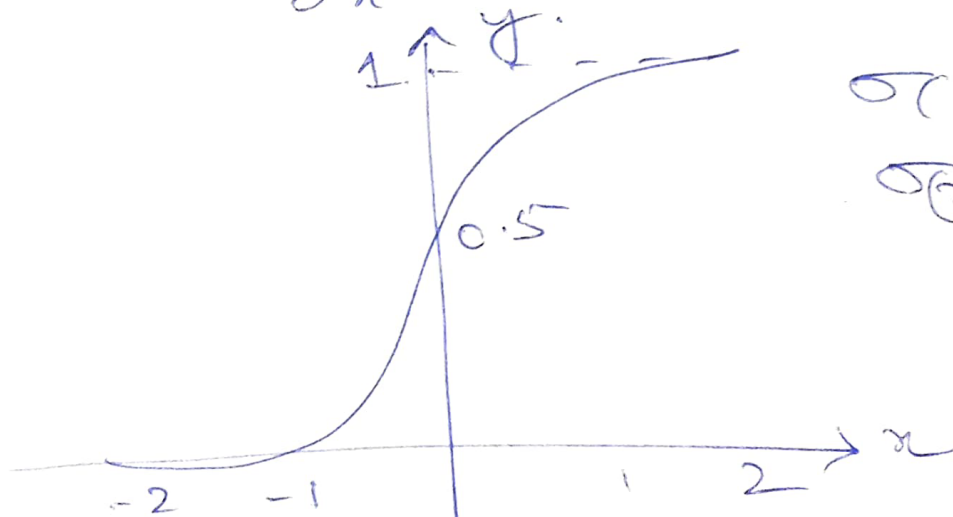
Is it because of Activation function?

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Interested in gradients, let us find the gradient of this function

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$$



$$\sigma(x) = 1 \text{ (at } -\infty \text{)}$$

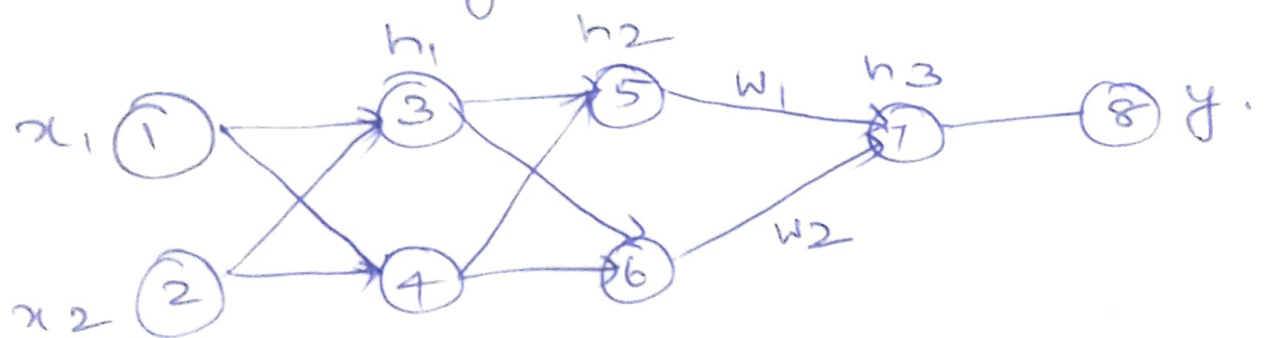
$$\sigma(x) = 0 \text{ (at } \infty \text{)}$$

Saturated neurons thus cause gradient to vanish.

What would happen if we use sigmoid function and initialize weights to very high values? (7)

→ Neurons will saturate very quickly.

→ The gradients will vanish and the training will stall.



$$\nabla_{w_1} = \frac{\partial \text{MSE}}{\partial o_8} \frac{\partial o_8}{\partial I_8} \frac{\partial I_8}{\partial o_7} \frac{\partial o_7}{\partial I_7} \frac{\partial I_7}{\partial w_1}.$$

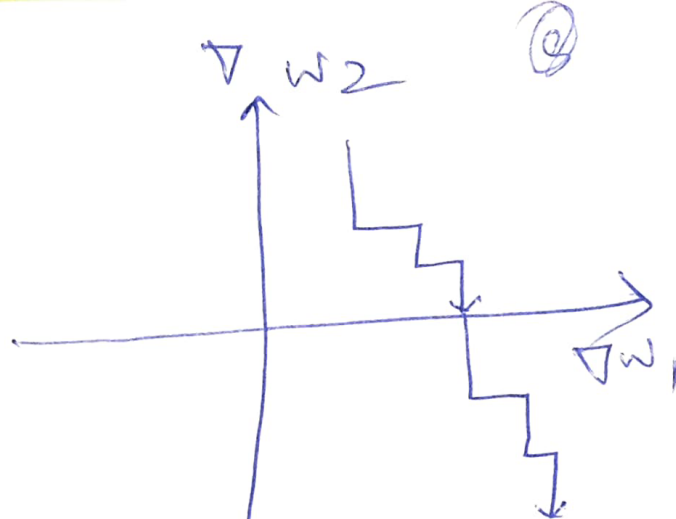
$$\nabla_{w_2} = \frac{\partial \text{MSE}}{\partial o_8} \frac{\partial o_8}{\partial I_8} \frac{\partial I_8}{\partial o_7} \frac{\partial o_7}{\partial I_7} \frac{\partial I_7}{\partial w_2}.$$

Consider $\frac{\partial I_7}{\partial w_1}$ and $\frac{\partial I_7}{\partial w_2}$ are

between $[0, 1]$.

If the first common term are positive then both ∇_{w_1} and ∇_{w_2} are positive, else ∇_{w_1} and ∇_{w_2} are negative.

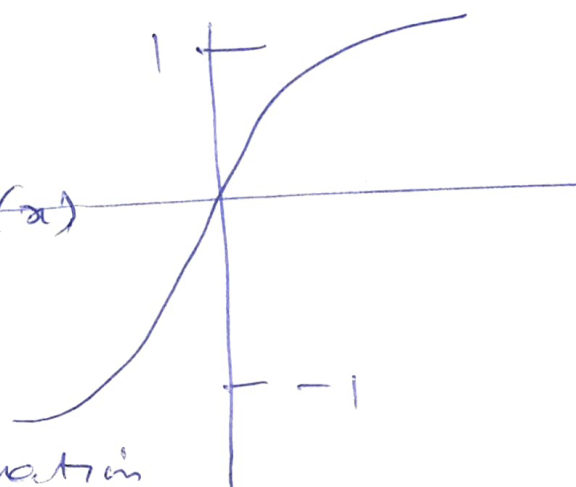
Sigmoidal
function



tanh(x)

Compress all input to the range $[-1, 1]$.

$$\frac{\partial \tanh(x)}{\partial x} = 1 - \tanh^2(x)$$



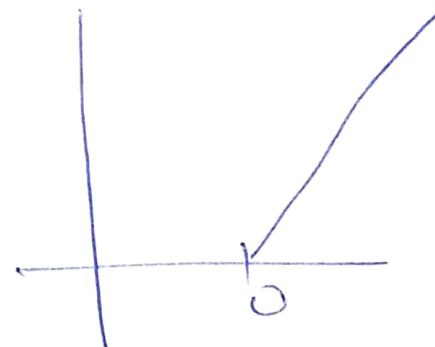
Gradient still

vanishes at saturation

Computationally expensive.

ReLU

$$f(x) = \max(0, x)$$



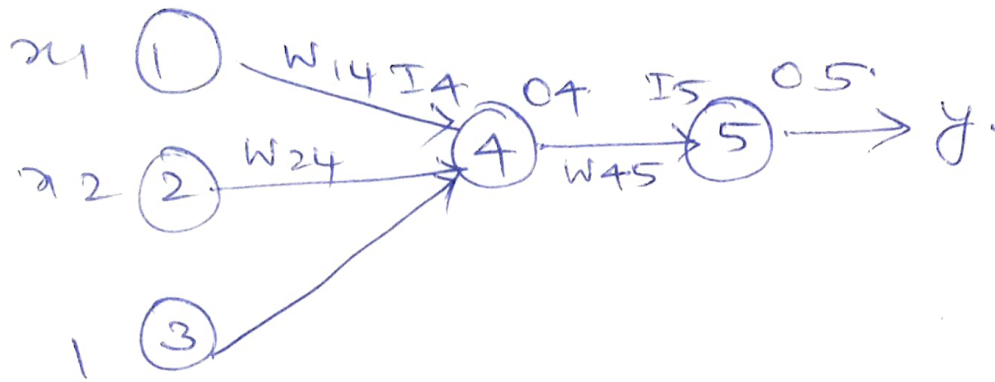
→ Non-linear function.

→ Doesn't saturate in the positive region

→ Computationally Efficient.

→ Converges much faster than Sigmoid/tanh.

9



$$\frac{\partial \text{ReLU}(x)}{\partial x} = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$

(ie) $w_{14}x_1 + w_{24}x_2 + b \leq 0$

$$\frac{\partial J_W}{\partial w_{14}} = \frac{\partial J_W}{\partial o_5} \frac{\partial o_5}{\partial I_5} \frac{\partial I_4}{\partial o_4} \frac{\partial o_4}{\partial I_4} \frac{\partial I_4}{\partial w_{14}}$$

↓
0 → leads entire expression to be zero.

In practice a large fraction of ReLU can die if the learning rate is set too high.

Leaky ReLU / Parametric ReLU

- NO saturation
- will not die.

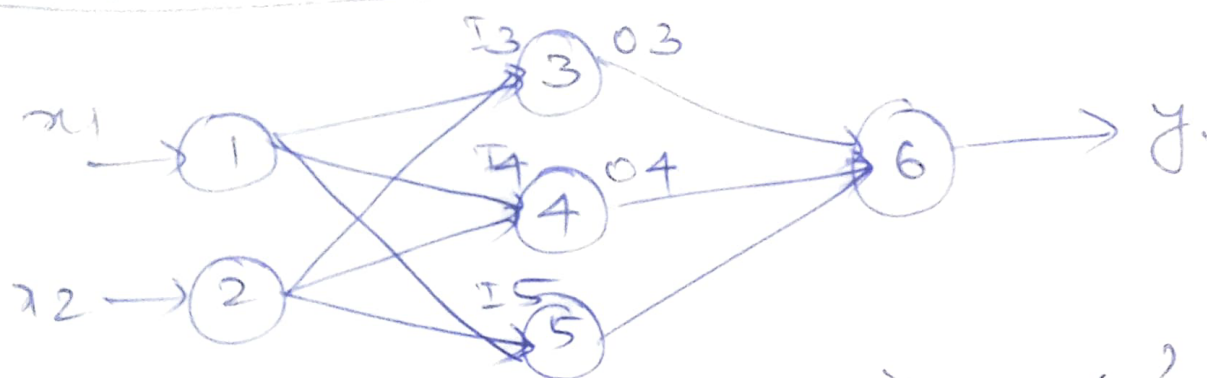
$$f(x) = \max(0, 0.01x, x)$$

Exponential Linear Unit

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ a e^x - 1 & \text{if } x \leq 0 \end{cases}$$

expensive

Weight Initialization Strategies



Initialize all weights to 'zero'?

$$I_3 = x_1 w_{13} + x_2 w_{23}$$

$$I_4 = x_1 w_{14} + x_2 w_{24}$$

$$\therefore I_3 = I_4 = 0$$

$$\therefore 0_3 = 0_4$$

What happens during back propagation

$$\nabla w_{13} = \frac{\partial I_w}{\partial 0_6} \frac{\partial 0_6}{\partial I_6} \frac{\partial I_6}{\partial 0_3} \frac{\partial 0_3}{\partial I_3} \frac{\partial I_3}{\partial w_{13}}$$

$$\nabla w_{14} = \frac{\partial I_w}{\partial 0_6} \frac{\partial 0_6}{\partial I_6} \frac{\partial I_6}{\partial 0_4} \frac{\partial 0_4}{\partial I_4} \frac{\partial I_4}{\partial w_{14}}$$

$$\therefore 0_3 = 0_4$$

$$I_3 = I_4$$

$$\therefore \nabla w_{13} = \nabla w_{14}$$

Thus both weights will get same update and remain equal.

This is true for w_{23} and w_{24} . (11)

∴ Known as symmetry breaking problem.

∴ This happens if all the weights in a network are initialized to the same value.